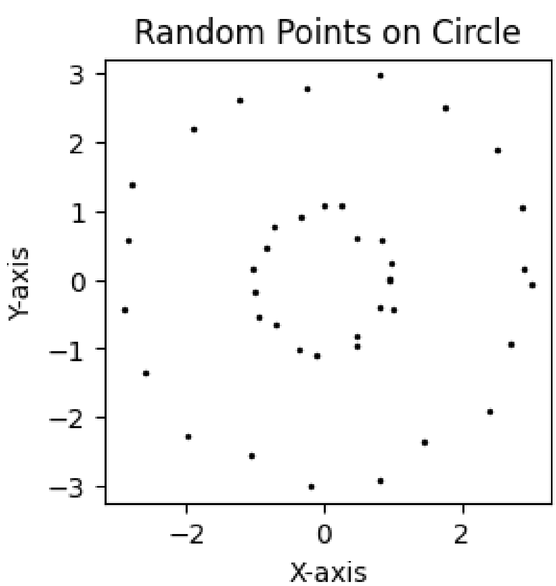
STAT672_Homework2_spectralclustering

February 11, 2025

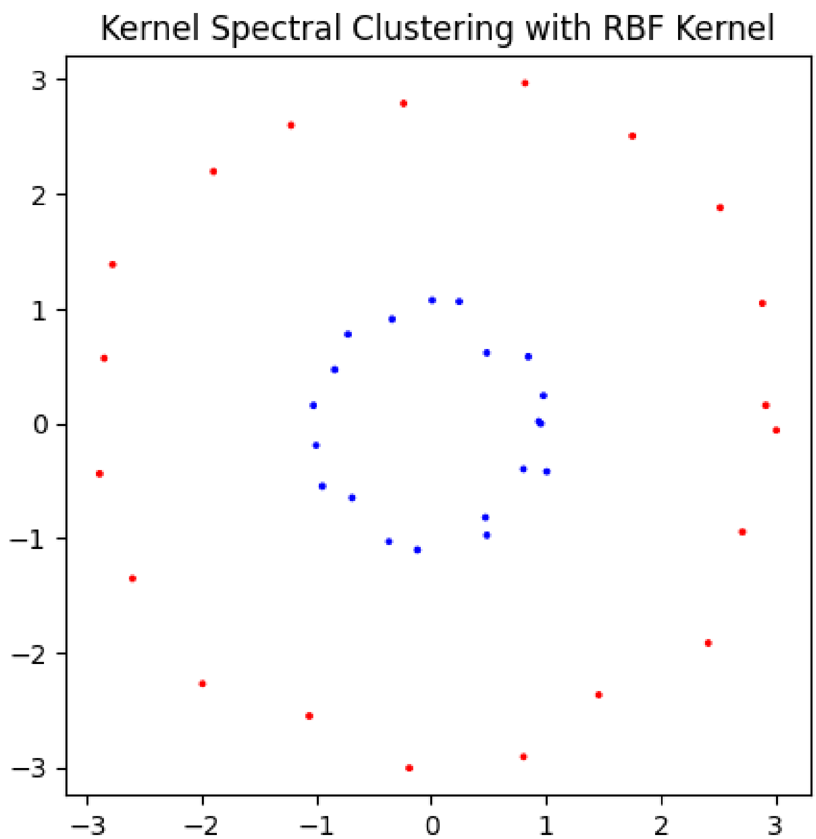
[101]:
```
# Loaded variable 'df' from URI: /Users/user/Desktop/kkm1.csv
import pandas as pd
import numpy as np
df = pd.read_csv(r'/Users/user/Desktop/kkm1.csv')
```

[115]:
```
x=np.array((df['V1'].values))
y=np.array((df['V2'].values))
X=np.transpose(np.vstack((x,y)))
```

[116]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(3, 3))
plt.axis('equal')  # To maintain aspect ratio
plt.title('Random Points on Circle')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.scatter(X[:,0],X[:,1],color='black',s=2)
plt.show()
```



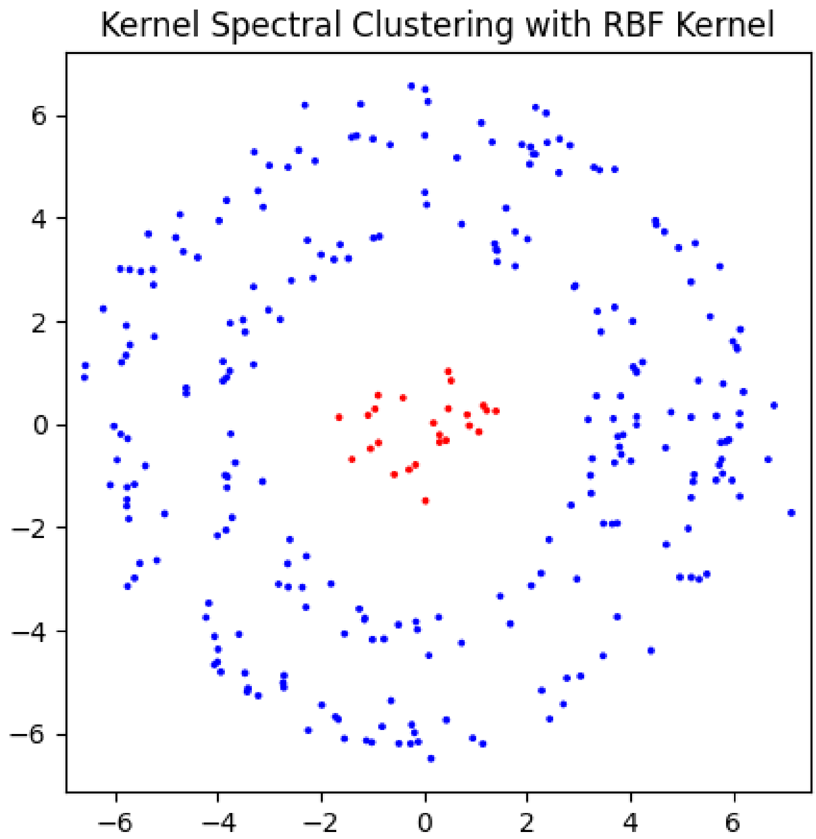
[117]:
```
from sklearn.cluster import SpectralClustering
# Apply kernel spectral clustering with RBF kernel
clustering = SpectralClustering(n_clusters=2,
                                affinity='rbf',
                                gamma=1.5,
                                random_state=0).fit(X)

# Visualize the results
plt.figure(figsize=(5, 5))
plt.scatter(X[:, 0], X[:, 1], c=clustering.labels_, cmap='bwr',s=3)
plt.title('Kernel Spectral Clustering with RBF Kernel')
plt.axis('equal')
plt.show()
```



[118]:
```
import math
#generate some random data
def generate_points_on_circle(radius, num_points,noise):
    xpoints=[]
    ypoints=[]
    np.random.seed(37)
```

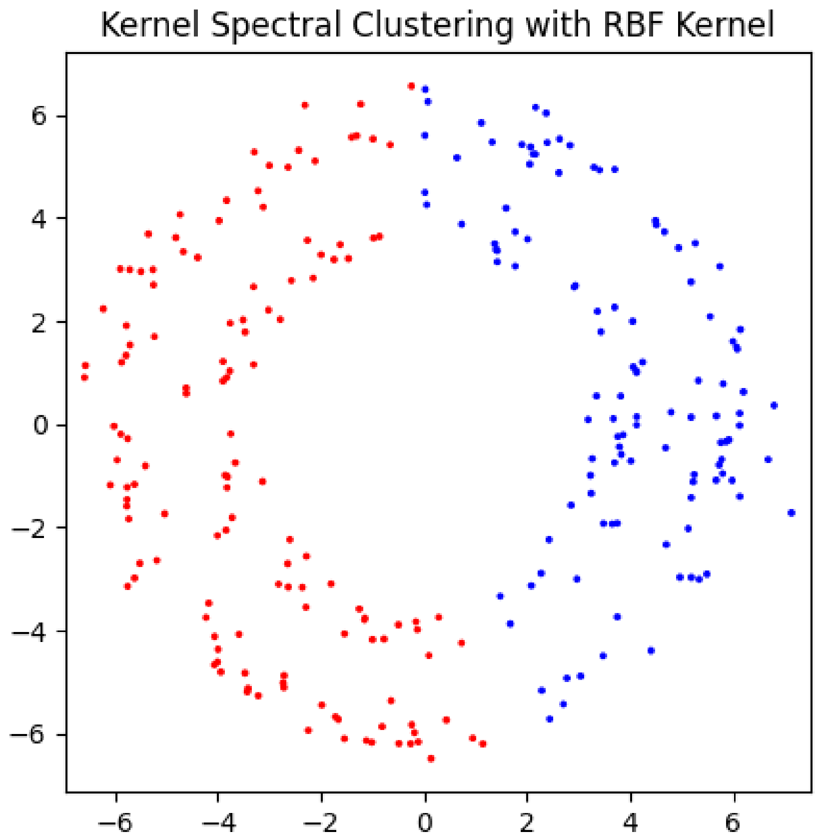
```
    for _ in range(num_points):
        # Generate a random angle in radians

        theta =np.random.uniform(0, 2 * math.pi)

        # Calculate x and y coordinates using polar coordinates

        x =  (radius+np.random.normal(0,noise,1)) * math.cos(theta)

        y =  (radius+np.random.normal(0,noise,1)) * math.sin(theta)

        xpoints.append(x)
        ypoints.append(y)

    return np.array(xpoints), np.array(ypoints)
#Parameteers
categories=3
factor=25
radius=[1,4,6]
noise=.4
#Contruct X(:,0) annd X(:,1)
Xp=np.empty((0,1))
Yp=np.empty((0,1))

for c in range(categories):
    num_points=factor*radius[c]
    xpoints,ypoints = generate_points_on_circle(radius[c],␣
 ↪int(num_points),noise)
    Xp=np.vstack((Xp,xpoints))
    Yp=np.vstack((Yp,ypoints))

X=np.hstack((Xp,Yp))
```

[119]:
```
from sklearn.cluster import SpectralClustering
# Apply kernel spectral clustering with RBF kernel
clustering = SpectralClustering(n_clusters=2,
                                affinity='rbf',
                                gamma=1.5,
                                random_state=0).fit(X)

# Visualize the results
plt.figure(figsize=(5, 5))
plt.scatter(X[:, 0], X[:, 1], c=clustering.labels_, cmap='bwr',s=3)
plt.title('Kernel Spectral Clustering with RBF Kernel')
plt.axis('equal')
plt.show()
```



[83]:
```
XX=X[np.where(clustering.labels_==0)[0],:]
```

[120]:
```
clustering = SpectralClustering(n_clusters=2,
                                affinity='rbf',
                                gamma=1,
                                random_state=1).fit(XX)

# Visualize the results
plt.figure(figsize=(5, 5))
plt.scatter(XX[:, 0], XX[:, 1], c=clustering.labels_, cmap='bwr',s=3)
plt.title('Kernel Spectral Clustering with RBF Kernel')
plt.axis('equal')
plt.show()
```


Kernel Spectral Clustering with RBF Kernel
6
4
2
0
−2
−4
−6
−6
−4
−2
0
2
4
6